

Astra Studio: AI 插件开发工作室

从业务洞察到生产级插件 —— 方法论驱动的插件开发流程

Version 0.2.0 | 2026 年 7 月

目录

- 概述
- 设计思想
- 方法论基础
- 架构总览
- 适用人群与场景
- 快速上手
- 规划流水线详解
- 领域专家体系
- 完整案例：儿童健康管理平台
- 最佳实践
- 附录

1. 概述

问题

当下构建 AI 智能体插件的过程是临时性的。团队从一个模糊的想法直接跳到编写 SKILL.md，缺乏结构化的分析过程。结果是：插件解决了错误的问题、技能边界不清晰、架构难以扩展。

解决方案

Astra Studio 是一个插件开发工作室，将结构化方法论引入插件开发的"外循环"：

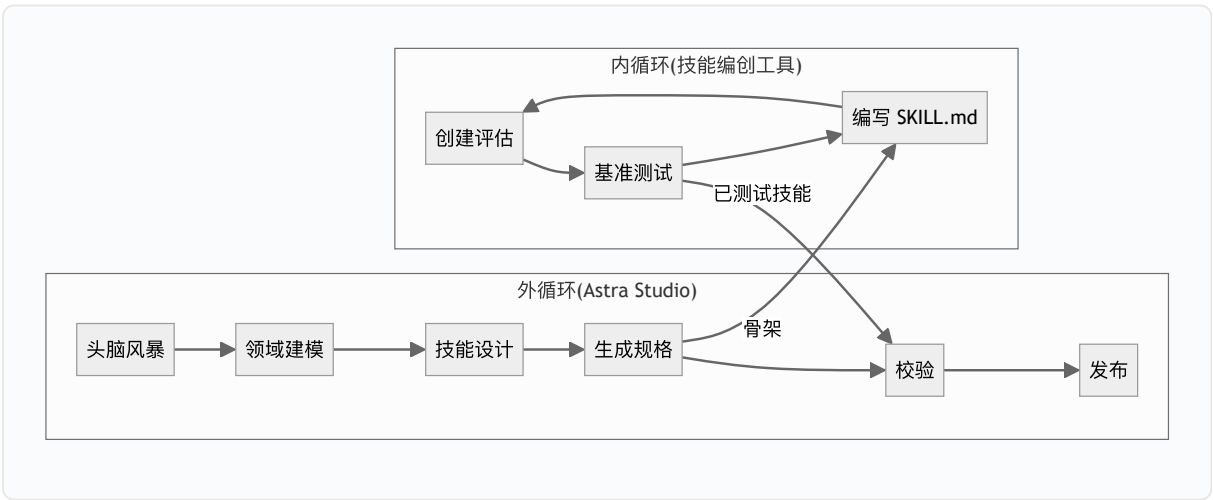
- 编码之前：多角色头脑风暴、用户研究、领域建模、机会评估
- 设计过程中：结构化技能拆分、数据流映射、复杂度评估
- 上线之前：自动化校验、清单合规、依赖完整性检查

核心差异

维度	没有 Astra Studio	有 Astra Studio
需求发现	"我觉得需要一个 X 插件"	多角色事件风暴 + 领域专家验证
用户理解	凭经验假设	结构化画像卡片 + 同理心地图 + 旅程地图
架构设计	直觉式技能拆分	数据流驱动的分解 + 复杂度分层
领域知识	通用猜测	动态领域专家咨询
质量保障	人工检查	自动化校验脚本
过程产出	只有代码	画像、旅程、流程图、领域画布、行为矩阵、机会评估

Astra Studio 不做什么

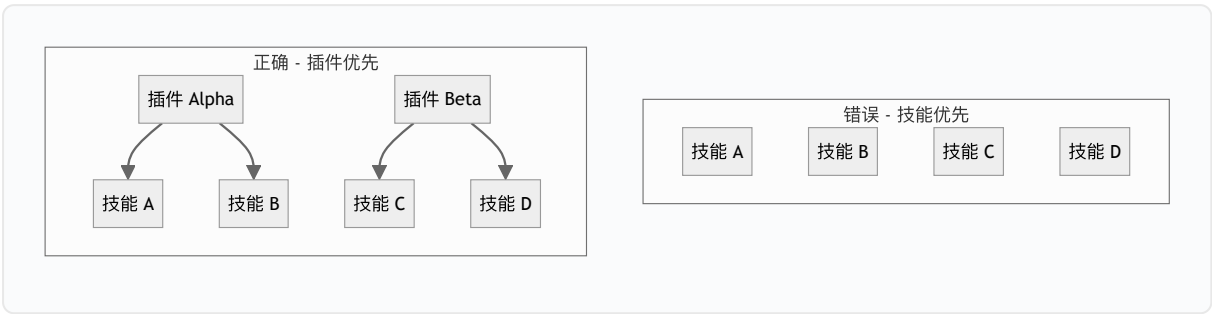
Astra Studio 负责外循环 —— 规划、设计、校验、发布。内循环（编写具体 SKILL.md 内容、创建评估用例、基准测试）由技能编创工具（如官方 `skill-creator`）完成。Astra Studio 产出骨架，编创工具填充内容。



2. 设计思想

2.1 插件优先架构

工作区以**插件**为粒度组织，而非技能。插件是安装的最小单位 —— 用户安装插件，不是单个技能。

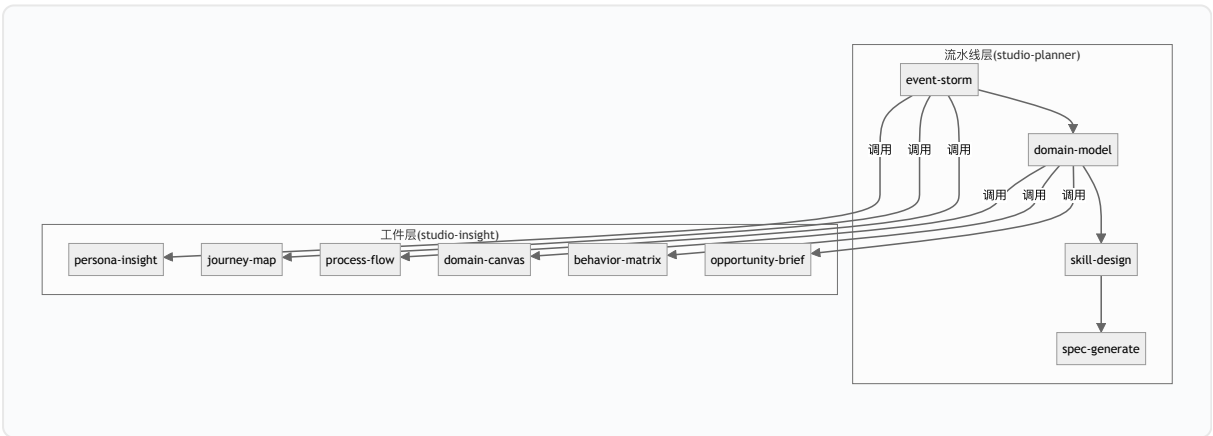


2.2 双层技能架构

Astra Studio 的技能分为两层：

层次	角色	可以独立运行？
流水线技能	编排一系列步骤	可以，但设计为串联使用
工件技能	产出独立的专业交付物	完全独立

流水线技能调用工件技能；工件技能永远不调用流水线技能。



2.3 Git 管理的开发工作区

`studio/` 目录位于用户项目中，**提交到 git**。它包含设计决策、论证过程和开发状态 —— 不仅仅是代码。

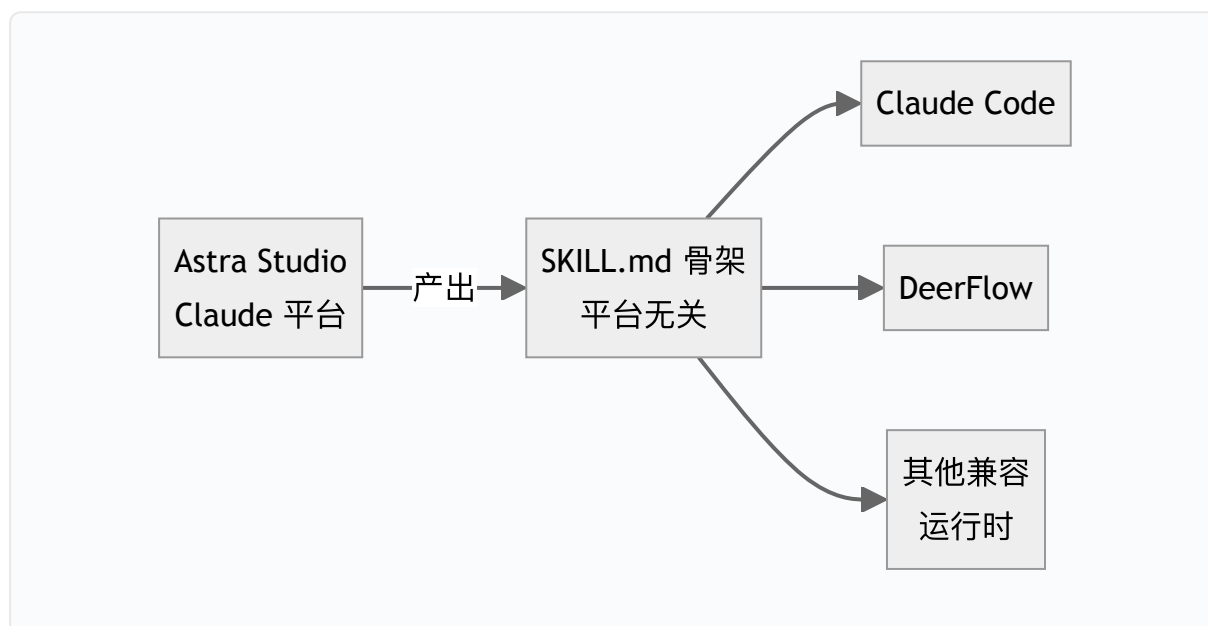
```
studio/
├─ config.yaml      # 共享配置
├─ agents/          # 自定义领域专家（团队共享）
```

```
|— changes/          # 活跃开发（画像、地图、骨架）
|— archive/         # 已发布记录（历史）
```

为什么要 git 管理？ 因为设计决策具有版本控制价值。当未来的开发者问"为什么这个技能要这样拆分？"时，答案就在 `studio/changes/{plugin}/skill-map.md` 里。

2.4 产出平台无关

Astra Studio 自身运行在 Claude Code 平台上。但它产出的插件（SKILL.md 骨架、清单、命令文件）是**平台无关**的——不包含任何 Claude 特定引用，可以在任何兼容的运行时报上执行。



2.5 专家范围：规划时 vs 运行时

领域专家分为两类：

阶段	存放位置	随插件分发？
规划时专家	<code>studio-insight/agents/</code> 或 <code>studio/agents/</code>	否 —— 知识被"消化"进 SKILL.md
运行时专家	<code>{target_dir}/agents/</code>	是 —— 随插件一起安装

规划时专家（产品经理、架构师等）参与**规划阶段**，他们的知识被编入 SKILL.md 的指令中。运行时专家（如营养顾问）在插件实际运行时被调用，需要随插件一起分发。

类比：建筑师参与设计房屋（规划时），但物业管理员需要住在小区里（运行时）。

`skill-design` 阶段的 `expert-scoped` 特征检测会自动区分这两类专家。

2.6 插件特征驱动脚手架

`skill-design` 阶段会自动检测 5 种跨领域特征 (traits)，驱动 `spec-generate` 产出条件性的脚手架：

特征	检测信号	下游影响
<code>stateful</code>	插件跨会话管理项目数据	生成 init-workspace skill + 运行时 config/status 模板
<code>hil-gated</code>	流程中有人工审批节点	skill 在审批边界拆分；骨架注入审批门段落
<code>kb-dependent</code>	skill 需要超出 references/ 的领域知识	标注 KB 集成需求
<code>multi-pipeline</code>	领域有 2+ 条独立业务流水线	为每条流水线生成编排命令
<code>expert-scoped</code>	运行时需要与规划阶段不同的领域专家	区分规划时/运行时专家，运行时专家随插件分发

无特征的插件走原有路径，不受影响。

2.7 初始填充而非最终构建

`build-skills` 产出可工作的初稿，而非完成品。用户需要用 `skill-creator` 进行迭代优化：

维度	初始填充 (build-skills)	迭代优化 (skill-creator)
目的	快速起步	精打细磨
产出	可工作的初稿	生产级实现
覆盖	核心逻辑 + 基本流程	边界情况 + 错误处理 + 评估

2.8 版本化发布

`promote` 是里程碑，不是终态。每次发布创建版本化快照 (v0.1 → v0.2)，设计文档被复制（不是移动）到归档，活跃工作区保留供下一轮迭代使用。

3. 方法论基础

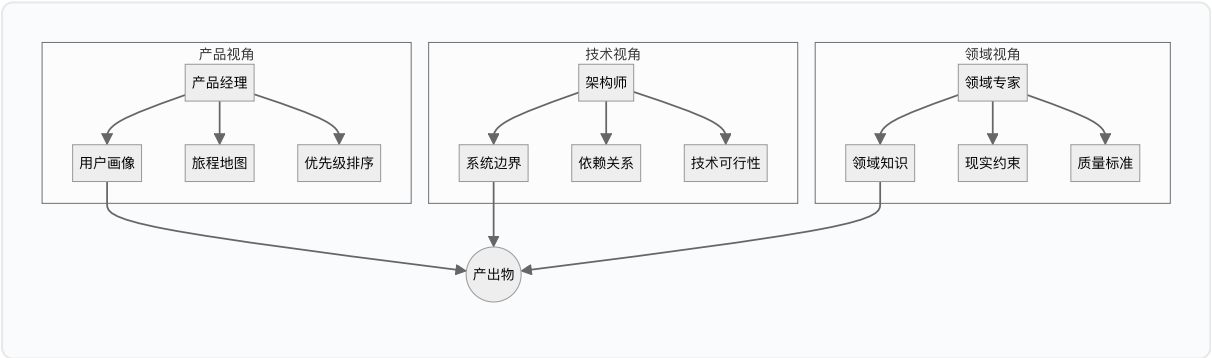
3.1 从事件风暴到插件架构

Astra Studio 借鉴了领域驱动设计（DDD）中的概念 —— 特别是事件风暴和战略设计 —— 并将其应用于插件开发。术语经过简化以提高可用性：

DDD 概念	Astra Studio 对应	通俗表达
事件风暴 (Event Storming)	event-storm	多角色头脑风暴
限界上下文 (Bounded Context)	业务域 (domain-canvas)	有明确边界的业务领域
上下文映射 (Context Map)	关系地图	领域之间如何交互
子域分类 (Subdomain Classification)	核心 / 支撑 / 通用	该自建还是外购
聚合 (Aggregate)	技能 (Skill)	功能的最小单元
通用语言 (Ubiquitous Language)	领域专家术语	从业者实际使用的词汇

3.2 多角色分析

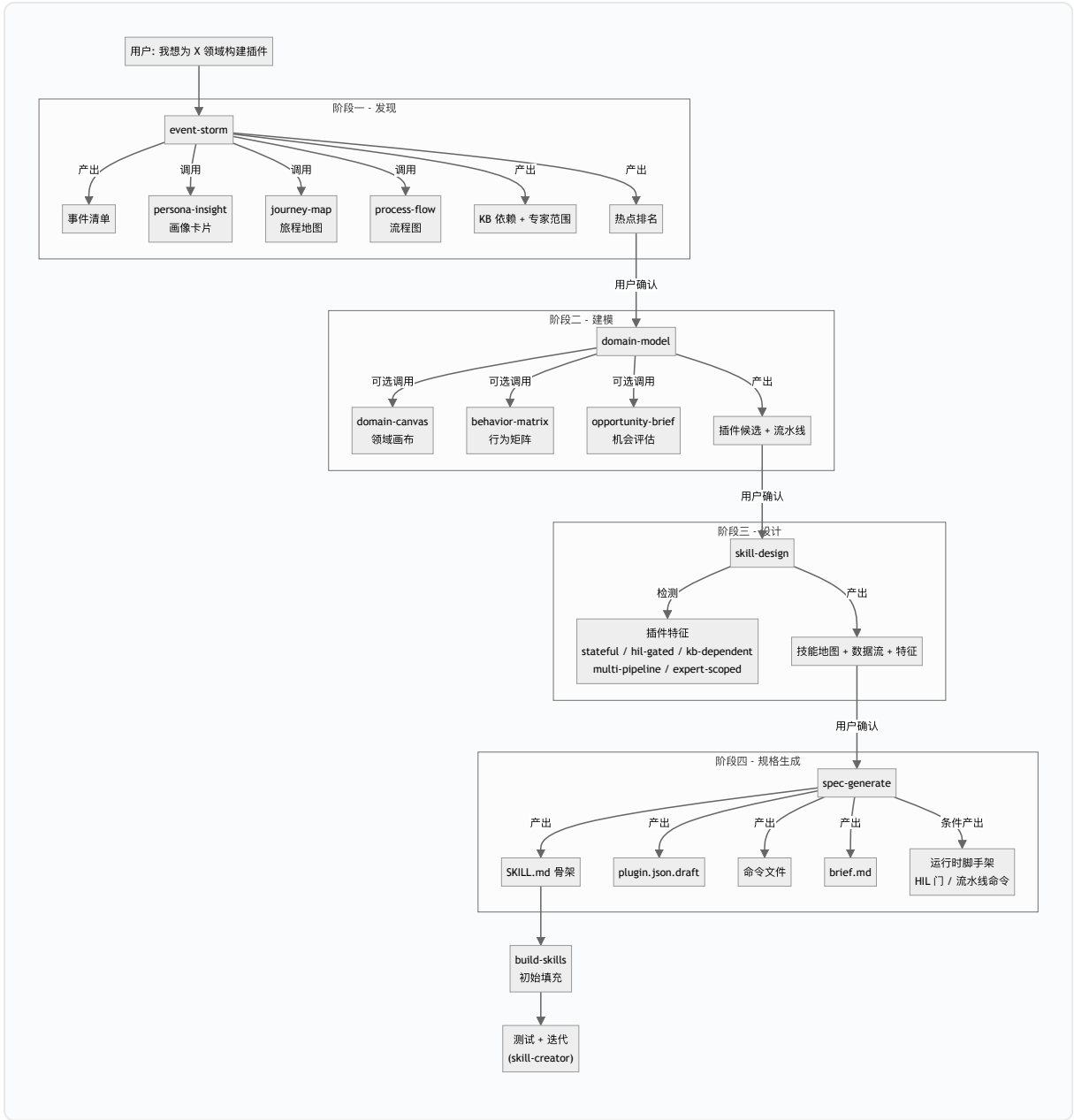
每个分析步骤都涉及多个视角：



这样可以避免： – **产品盲区**：为不存在的问题构建技术上优雅的解决方案 – **技术盲区**：设计不符合领域现实的方案 – **领域盲区**：遗漏只有从业者才知道的现实约束

3.3 规划流水线

完整的规划流程遵循结构化序列，每个阶段之间都有用户确认检查点：

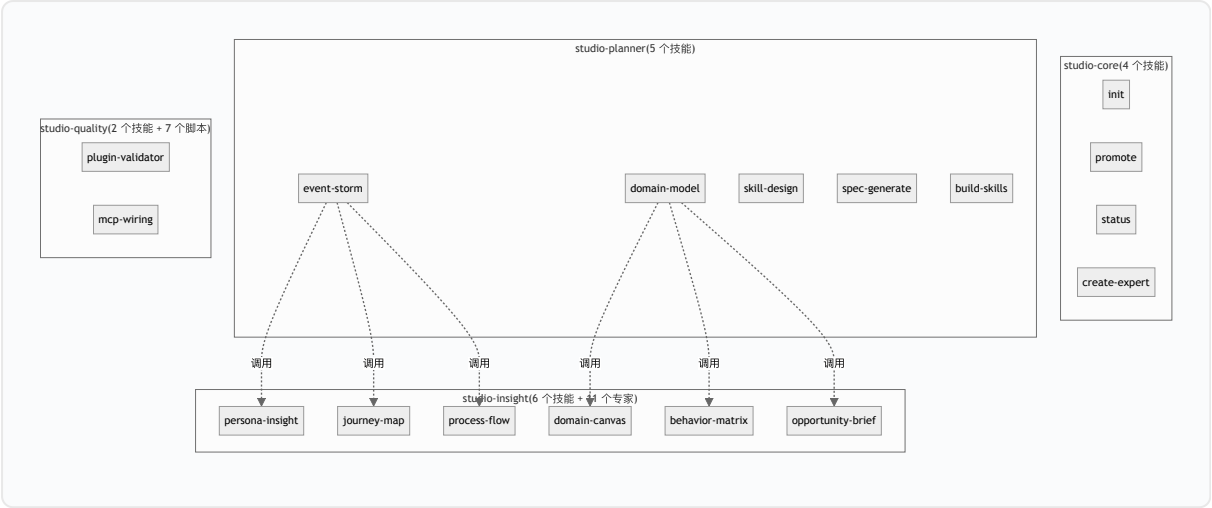


完整分析 vs 快速模式: `domain-model` 支持两种执行深度。完整分析（默认）调用 `domain-canvas`、`behavior-matrix`、`opportunity-brief`；快速模式跳过这三个工件技能，直接从事件聚类推导插件候选。

4. 架构总览

4.1 插件矩阵

工具链的核心四件套构成完整的规划闭环：

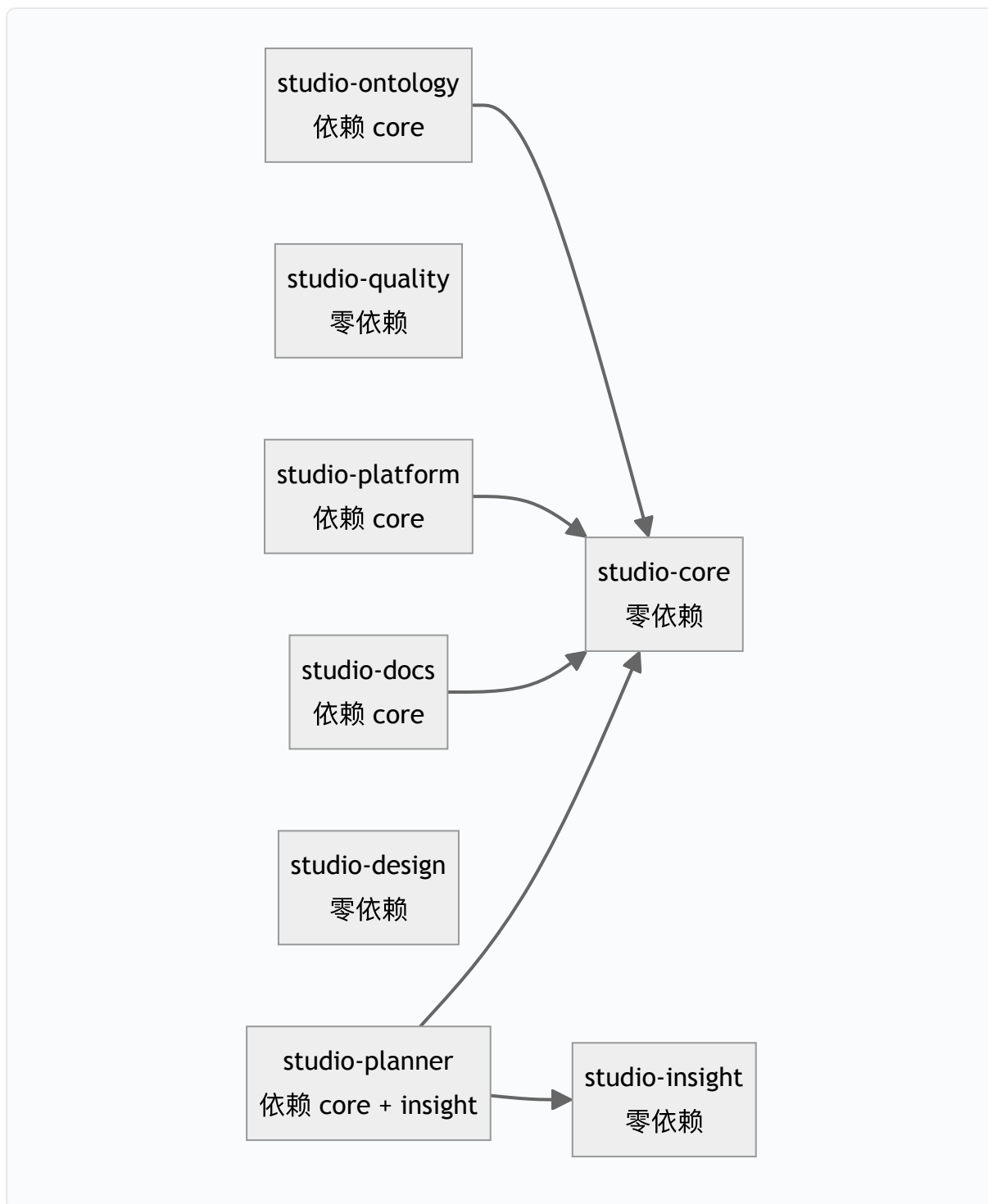


在核心四件套之上，有四个交付向工具链插件，各自消费规划产出物：

插件	技能数	消费什么	产出什么
studio-docs	6	规划产出物 + 文档蓝图	正式项目文档（方案/可研/招投标），支持 MD/DOCX/PDF
studio-platform	6	规划产出物	行业大模型平台文档套件（脑图、智能体映射、数仓/知识图谱/ML/RAG 设计、项目计划、.pen 可视化、迎检话术）
studio-design	3	UI 截图 / .pen 原型	Pencil 原型 → OpenSpec 提案 → 可运行代码
studio-ontology	3	事件风暴 / 行为矩阵 / 领域画布，或领域接入规格	可运行的 clife-onto-engine 本体插件（五元组：Object/Link/Function/Rule/Action）

此外仓库还分发一个垂直插件 `fund-review`（4 个技能 + 40 个 Python 脚本）—— 它不是工具链的一部分，而是工具链产出的领域插件，放在同一个 marketplace 里分发。它是本方法论的端到端验证：政府信息化采购的报价优化与财评预审，从事件风暴一路走到可安装插件。

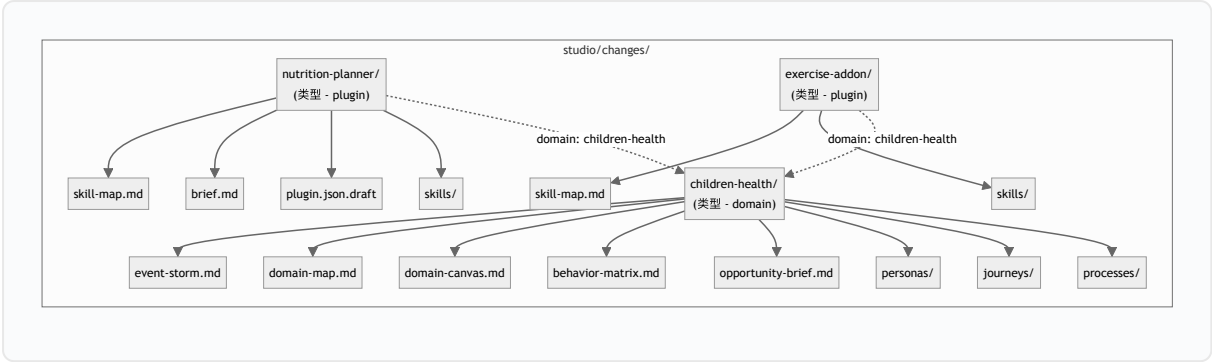
4.2 依赖关系



八个工具链插件中有三个**零依赖**（core、insight、quality）加 design，可以独立安装和使用。studio-docs / studio-platform / studio-ontology 只硬依赖 studio-core，规划产出物是软消费——没有也能跑，只是要手工补输入。

垂直插件 `fund-review` 零插件依赖，但需要 Python 环境（`openpyxl`、`python-docx`）；它自带 `.fund-review/` 运行态工作区，与服务插件开发的 `studio/` 相互独立。

4.3 两级工作区模型



- **域级工作区**：共享产出物（事件风暴、领域地图、画像、旅程、流程）。一个域可以衍生多个插件。
- **插件级工作区**：插件专有设计文档（技能地图、brief.md、plugin.json.draft）。通过 status.json 中的 `domain` 字段引用父级域，`target_dir` 字段指向实现目录。不包含 SKILL.md 或命令文件 —— 这些直接在 `{target_dir}/` 中生成。

当插件具有特定特征时，`spec-generate` 会在 `{target_dir}/` 中产出额外脚手架：

- **stateful**：`skills/init-workspace/SKILL.md` + `templates/runtime-config.yaml.tpl` + `templates/runtime-status.json.tpl`
- **hil-gated**：相关 SKILL.md 骨架中注入 `## Approval Gate` 段落
- **multi-pipeline**：每条流水线的 `commands/{pipeline-name}.md` 编排命令
- **expert-scoped**：`agents/` 目录中的运行时专家定义

4.4 技能全景

插件	技能	类型	主导角色	产出物
studio-core	init	管理	—	<code>studio/</code> 目录
studio-core	promote	管理	—	生产插件 + 归档
studio-core	status	管理	—	仪表板输出
studio-core	create-expert	管理	—	<code>studio/agents/{name}.md</code>
studio-insight	persona-insight	工件	产品经理	画像卡片 + 同理心地图
studio-insight	journey-map	工件	产品经理	旅程地图 + 情绪曲线

插件	技能	类型	主导角色	产出物
studio-insight	process-flow	工件	架构师	流程图 + 决策点
studio-insight	domain-canvas	工件	架构师	领域边界地图
studio-insight	behavior-matrix	工件	架构师	行为矩阵
studio-insight	opportunity-brief	工件	产品经理	优先级排序 + 投入评估
studio-planner	event-storm	流水线	多角色	event-storm.md + 工件调用 + KB 依赖 + 专家范围
studio-planner	domain-model	流水线	架构师	domain-map.md + 插件候选（支持完整/快速模式）
studio-planner	skill-design	流水线	架构师	skill-map.md + 特征检测 + 依赖图
studio-planner	spec-generate	流水线	—	SKILL.md 骨架 + 清单 + 特征条件脚手架
studio-planner	build-skills	流水线	—	初始填充（调用 skill-creator 产出可工作的初稿）
studio-quality	plugin-validator	质量	—	校验报告
studio-quality	mcp-wiring	质量	—	.mcp.json 配置
studio-docs	define-blueprint	交付	—	文档蓝图（结构 + 合规规则 + 导出格式）
studio-docs	create-doc-expert	交付	—	写作专家定义（风格 + 术语 + 引用习惯）
studio-docs	plan-document	交付	—	章节映射 + 并行写作策略
studio-docs	write-section	交付	写作专家	单章正文（支持断点恢复）

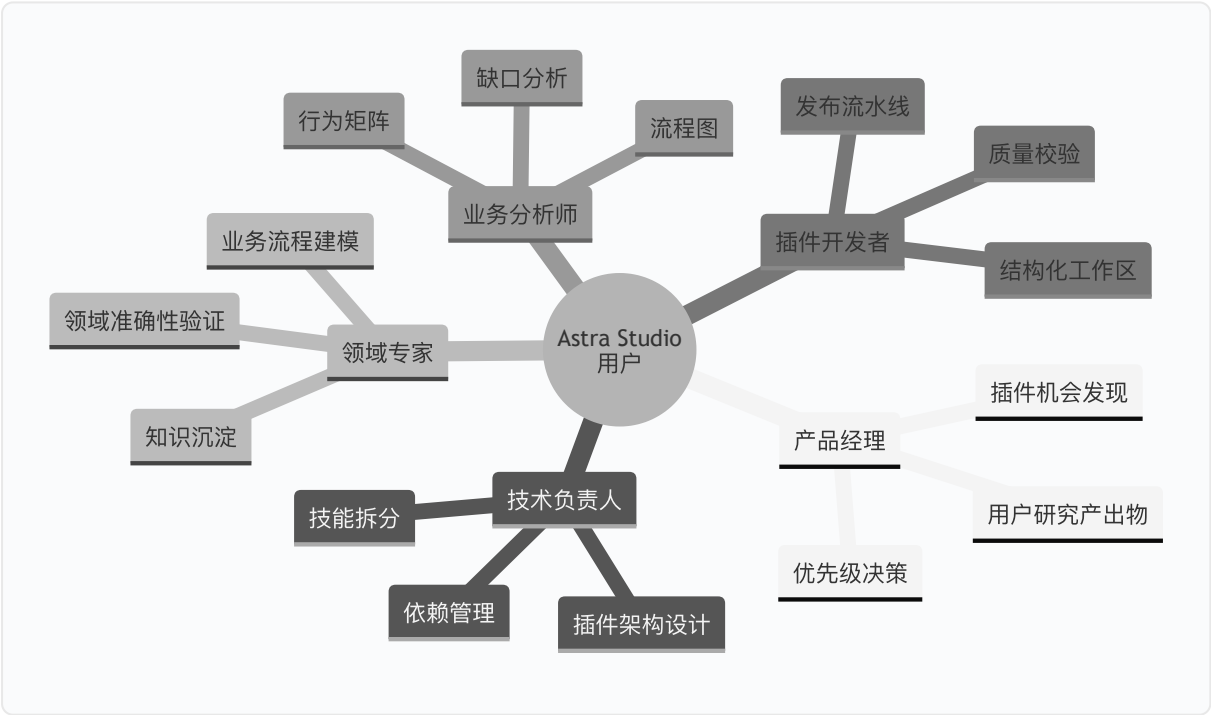
插件	技能	类型	主导角色	产出物
studio-docs	assemble-document	交付	—	合稿文档 + 质量报告
studio-docs	export-document	交付	—	DOCX / PDF (Mermaid 渲染 + 公文排版)
studio-platform	generate-brainmap	交付	—	脑图索引 + 分模块智能体详情
studio-platform	generate-agent-mapping	交付	架构师	智能体 ↔ 插件 ↔ 技能映射 + 架构图
studio-platform	generate-tech-designs	交付	架构师	数仓 / 采集 / 知识图谱 / ML / RAG 五份设计
studio-platform	generate-project-plan	交付	—	并行开发轨 + 甘特图 + 里程碑 + 风险
studio-platform	generate-platform-visual	交付	—	四帧 .pen 可视化
studio-platform	generate-speech	交付	—	迎检 / 汇报话术
studio-design	screenshot-to-pencil	设计	—	.pen 原型
studio-design	pencil-to-openspec	设计	—	OpenSpec 变更提案
studio-design	openspec-to-code	设计	—	React / Next.js 代码
studio-ontology	model	FDE	本体架构师	五元组本体 IR
studio-ontology	map	FDE	规则工程师	数据源 → 本体映射 + 规则分诊
studio-ontology	compile	FDE	—	可运行 clife-onto-engine 插件 (schema YAML + handler 骨架 + CQ)

垂直插件 `fund-review` 的技能（不属于工具链，列此对照）：

插件	技能	类型	主导角色	产出物
fund-review	init-session	运行时	—	<code>.fund-review/{session-id}/</code> (standard.json + quote.json)
fund-review	quote-optimize	运行时	报价优化师	optimize-suggestions.json（带标准出处，等人工审批）
fund-review	quote-rewrite	运行时	—	quote-final.xlsx + 变更日志
fund-review	quote-review	运行时	财评审核员	评分报告（每条结论配标准页码 + 章节）

5. 适用人群与场景

5.1 目标用户



5.2 使用场景

场景 A：全新插件开发

情况：团队想为一个尚未系统化的业务领域构建插件。

流程：完整流水线 —— event-storm → domain-model → skill-design → spec-generate

核心价值：通过在编码前投入理解，避免构建错误的东西。

场景 B：独立业务分析

情况：产品经理需要用户旅程地图或画像卡片，但目前不是在构建插件。

流程：直接调用工件技能 —— `/studio-insight:persona-insight` 、 `/studio-insight:journey-map`

核心价值：无需完整流水线即可获得专业的业务分析产出物。

场景 C：已有插件校验

情况：团队有临时构建的插件，想在发布前验证其结构。

流程：`/studio-quality:validate {path}` —— 运行结构、技能、依赖检查。

核心价值：在用户遇到问题之前捕获清单错误、缺失字段和断裂引用。

场景 D：领域专家引入

情况：团队进入一个新领域（如医疗健康），需要为 AI 智能体沉淀领域知识。

流程：`/studio-core:create-expert` 将领域专家视角形式化。专家随后自动参与所有后续分析。

核心价值：领域知识变得可复用、团队共享、版本可追溯。

场景 E：多团队插件生态

情况：组织内多个团队为相关领域构建插件。

流程：每个团队运行完整流水线。域级产出物（事件风暴、领域画布）提供共享上下文。`studio/agents/` 中的自定义专家确保一致性。

核心价值：跨团队一致的方法论 + 共享的领域知识。

6. 快速上手

6.1 安装

```
# 注册 marketplace
claude plugin marketplace add github:Ameng2001/astra-studio-plugins
```

```
# 安装核心四件套
claude plugin install studio-core@astra-studio
claude plugin install studio-insight@astra-studio
claude plugin install studio-planner@astra-studio
claude plugin install studio-quality@astra-studio

# 按需加装交付向插件
claude plugin install studio-docs@astra-studio
claude plugin install studio-platform@astra-studio
claude plugin install studio-design@astra-studio
claude plugin install studio-ontology@astra-studio
```

6.2 初始化项目

```
# 在你的项目目录中
/studio-core:init
```

创建的目录：

```
studio/
├─ config.yaml      # 配置
├─ agents/          # 自定义领域专家
├─ changes/         # 活跃开发
└─ archive/         # 已发布记录
```

6.3 三种使用方式

方式一：完整流水线（推荐用于新项目）

```
/studio-planner:plan "你的业务领域"
# → 逐步走完全部 4 个阶段，每个阶段有确认检查点
```

方式二：单独工件（用于获取特定交付物）

```
/studio-insight:persona-insight "目标用户描述"
/studio-insight:journey-map "用户工作流程"
/studio-insight:process-flow "业务流程"
/studio-insight:domain-canvas "你的领域"
/studio-insight:behavior-matrix "你的领域"
/studio-insight:opportunity-brief "你的领域"
```

方式三：仅质量检查（用于已有插件）

```
/studio-quality:validate path/to/your/plugin
/studio-quality:wire-mcp path/to/your/plugin
```

6.4 命令速查表

命令	功能
/studio-core:init	初始化工作区
/studio-core:status	显示开发仪表板
/studio-core:create-expert	创建/定制领域专家
/studio-core:promote {name}	创建版本化里程碑 (v0.1 → v0.2)
/studio-planner:plan {domain}	运行完整规划流水线
/studio-insight:persona-insight	生成用户画像卡片
/studio-insight:journey-map	生成用户旅程地图
/studio-insight:process-flow	生成业务流程图
/studio-insight:domain-canvas	生成领域边界地图
/studio-insight:behavior-matrix	生成行为矩阵
/studio-insight:opportunity-brief	生成机会评估
/studio-quality:validate {path}	校验插件结构
/studio-quality:wire-mcp {path}	配置 MCP 连接
/studio-docs:generate {plugin}	从规划产出物生成正式文档
/studio-docs:blueprint	定义/定制文档蓝图
/studio-docs:writer	创建写作专家
/studio-docs:export {file}	导出 DOCX / PDF
/studio-platform:platform-docs {domain}	生成平台文档套件 (6 步)
/studio-design:design	截图 → 原型 → 提案 → 代码全链
/studio-ontology:model {domain}	建五元组本体 IR
/studio-ontology:map	数据源映射 + 规则分诊

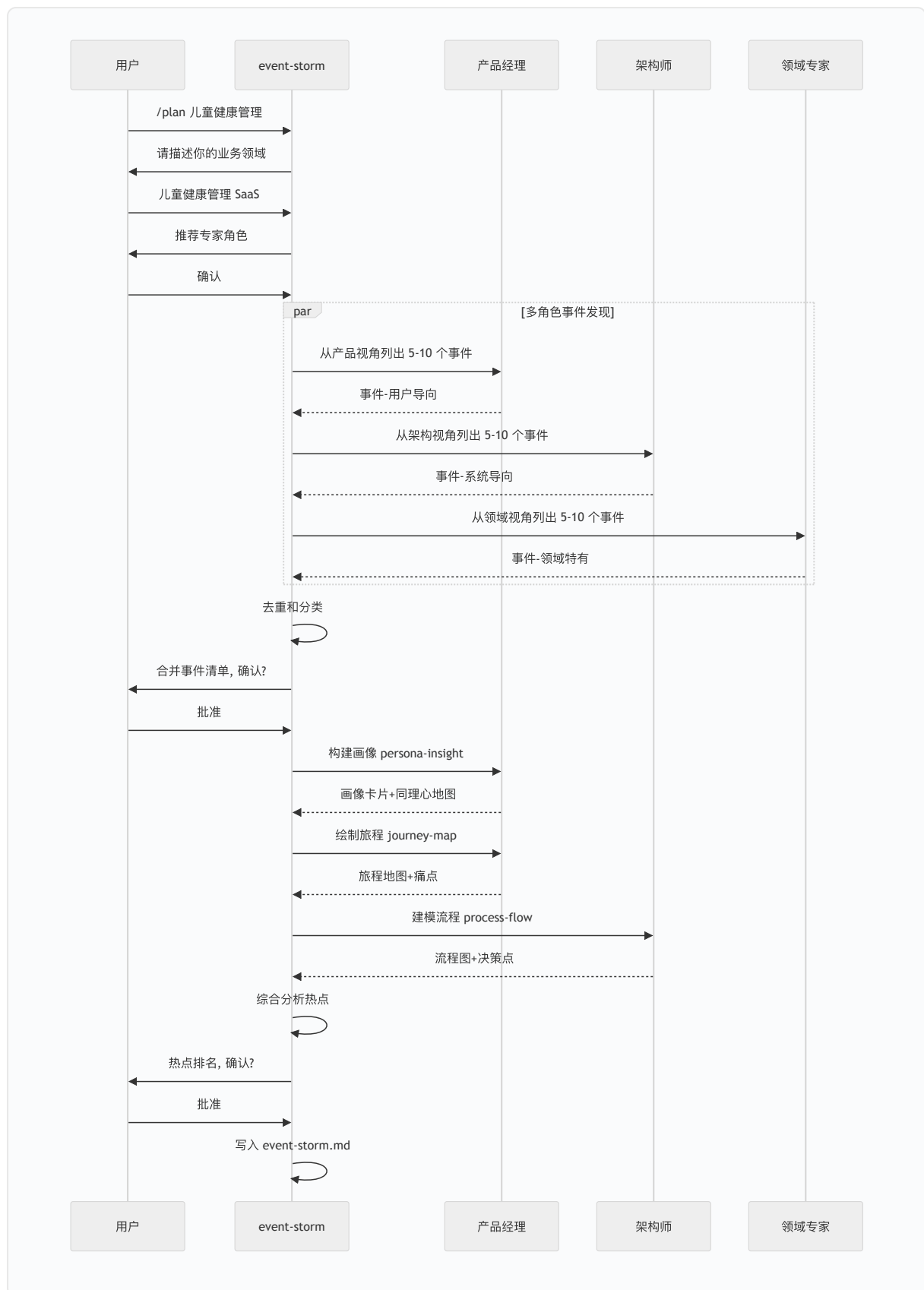
命令	功能
<code>/studio-ontology:compile</code>	编译成可运行本体插件
<code>/studio-ontology:validate</code>	校验本体一致性

7. 规划流水线详解

7.1 阶段一：事件风暴

目标：发现业务领域中真正发生的事情。

参与者：产品经理 + 架构师 + 2-4 位领域专家



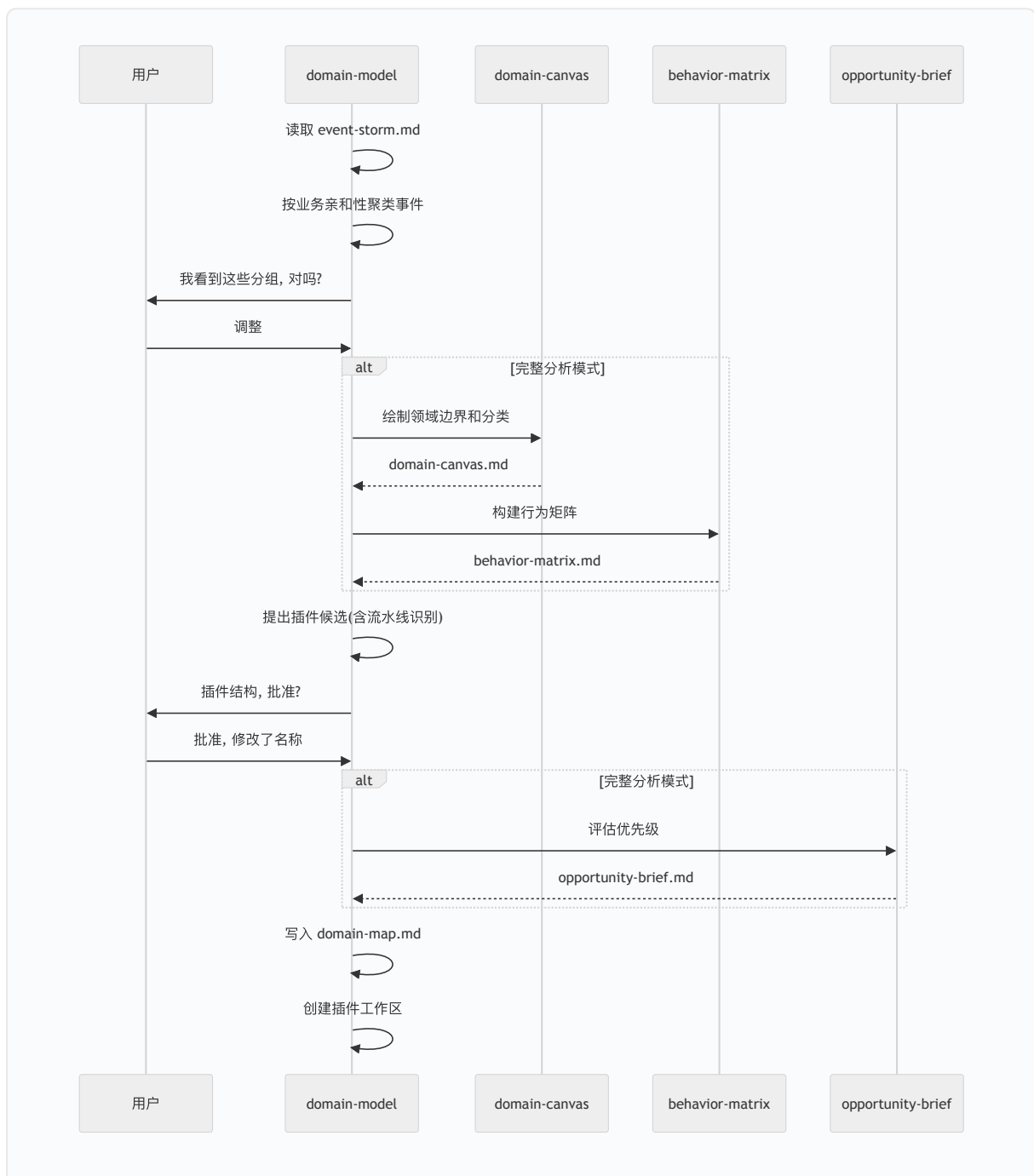
关键产出： - **event-storm.md** —— 综合头脑风暴结果 + KB 依赖分析 + 专家范围分析 - **personas/{name}.md** —— 画像卡片 + 同理心地图 - **journeys/{scenario}.md** —— 旅程地图 + 情绪曲线 - **processes/{process}.md** —— 业务流程图

新增：event-storm 阶段现在还会识别知识库依赖热点（哪些领域知识超出 LLM 能力范围）和专家范围（哪些专家是规划时使用、哪些需要在运行时随插件分发）。

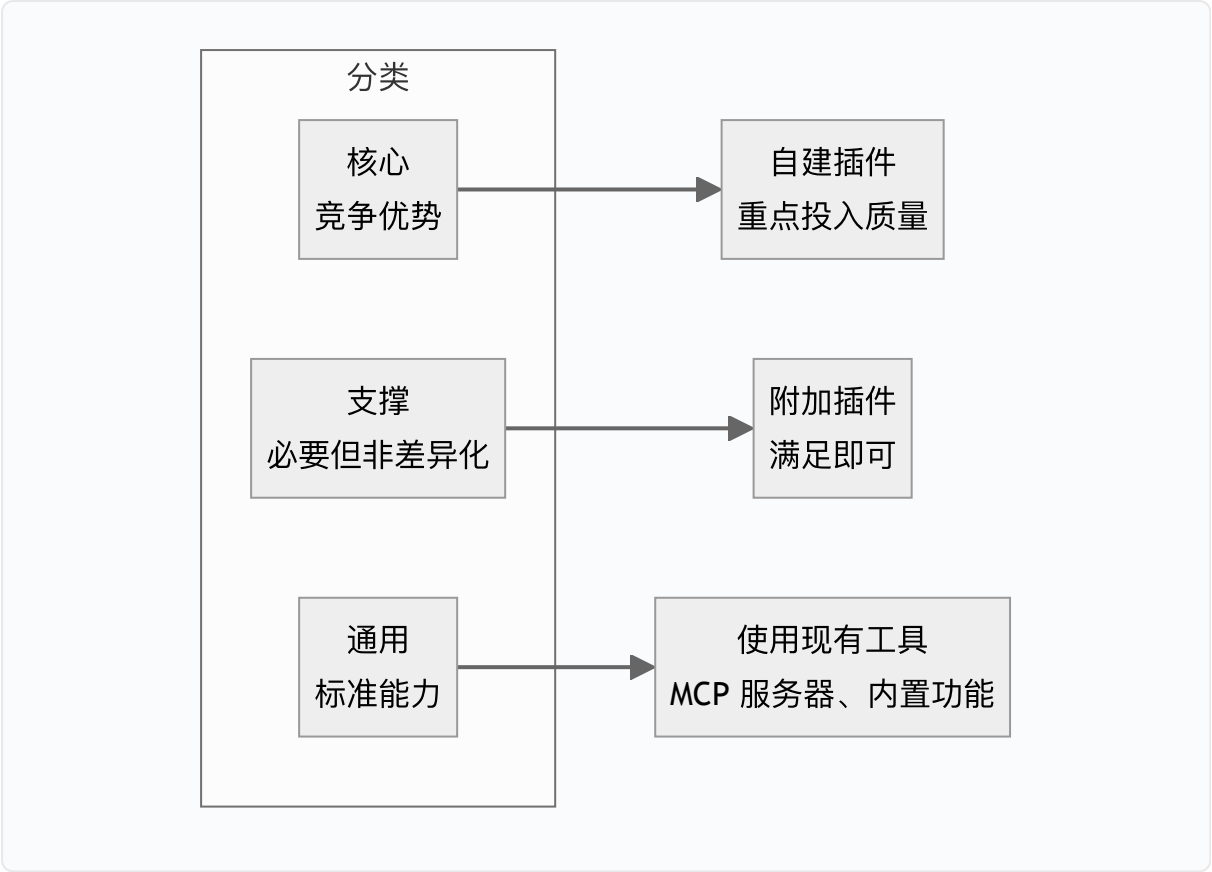
7.2 阶段二：领域建模

目标：识别插件边界，确定构建优先级。

执行深度：用户可选择两种模式： – **完整分析**（默认）：调用 domain-canvas、behavior-matrix、opportunity-brief，适合复杂领域或首次规划 – **快速模式**：跳过这三个工件技能，直接从事件聚类推导插件候选，适合领域已明确或迭代更新



关键决策：领域分类决定构建策略：



7.3 阶段三：技能设计

目标：将每个插件拆分为接口清晰、数据流明确的技能，并检测插件的跨领域特征。

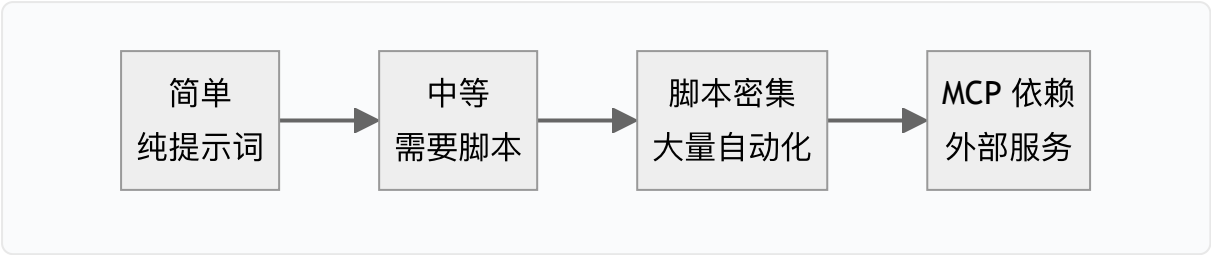
核心原则：

原则	指导方针
单一职责	每个技能只做一件事
用户决策点	在用户需要做决定的地方拆分
HIL 检查点	在人工审批节点拆分
流水线独立	不同业务流水线的技能不强行合并
明确接口	每个技能都有定义好的输入、输出和范围外
无循环依赖	数据单向流动

插件特征检测：

在技能拆分之前，`skill-design` 会从上游产出物（`event-storm.md`、`domain-map.md`）中检测 5 种跨领域特征：`stateful`、`hil-gated`、`kb-dependent`、`multi-pipeline`、`expert-scoped`。特征检测结果写入 `skill-map.md` 的 `## Plugin Traits` 段落，驱动 `spec-generate` 的条件性脚手架生成。

复杂度分层：



7.4 阶段四：规格生成

目标：生成全部规格文件 —— 无需人工输入。根据插件特征产出条件性脚手架。

基础生成内容（所有插件）：

文件	说明	平台属性
<code>brief.md</code>	从事件风暴综合的业务上下文	—
<code>plugin.json.draft</code>	插件清单	标准规范
<code>skills/{name}/SKILL.md</code>	技能骨架	平台无关
<code>commands/{name}.md</code>	命令文件	标准规范

特征条件生成（仅检测到特征时）：

特征	额外产出
<code>stateful</code>	<code>skills/init-workspace/SKILL.md</code> + <code>templates/runtime-config.yaml.tpl</code> + <code>templates/runtime-status.json.tpl</code>
<code>hil-gated</code>	相关 SKILL.md 骨架中的 <code>## Approval Gate</code> 段落
<code>multi-pipeline</code>	每条流水线的 <code>commands/{pipeline-name}.md</code> 编排命令
<code>expert-scoped</code>	<code>agents/</code> 目录中的运行时专家定义

此阶段完成后，状态从 `planning` 推进到 `building`。

7.5 阶段五：初始填充

目标：`build-skills` 调用 `skill-creator` 为每个骨架产出可工作的初稿。

这是初始填充，不是最终构建。初稿覆盖核心逻辑和基本流程，但边界情况、错误处理和评估用例需要用户后续用 `skill-creator` 迭代优化。

此阶段完成后，用户应： 1. 用真实输入测试每个技能 2. 用 `skill-creator` 迭代优化 3. 运行 `/studio-quality:validate` 校验 4. 运行 `/studio-core:promote` 创建版本化里程碑

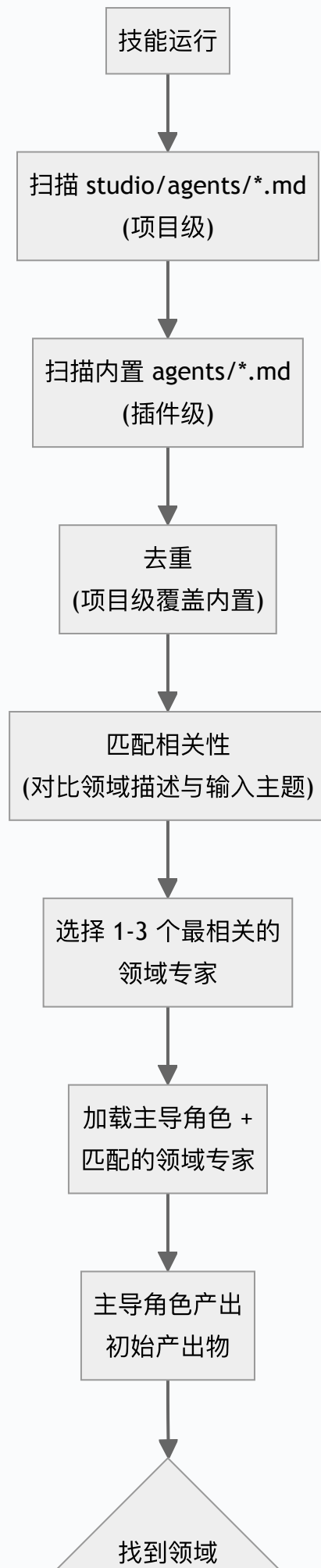
8. 领域专家体系

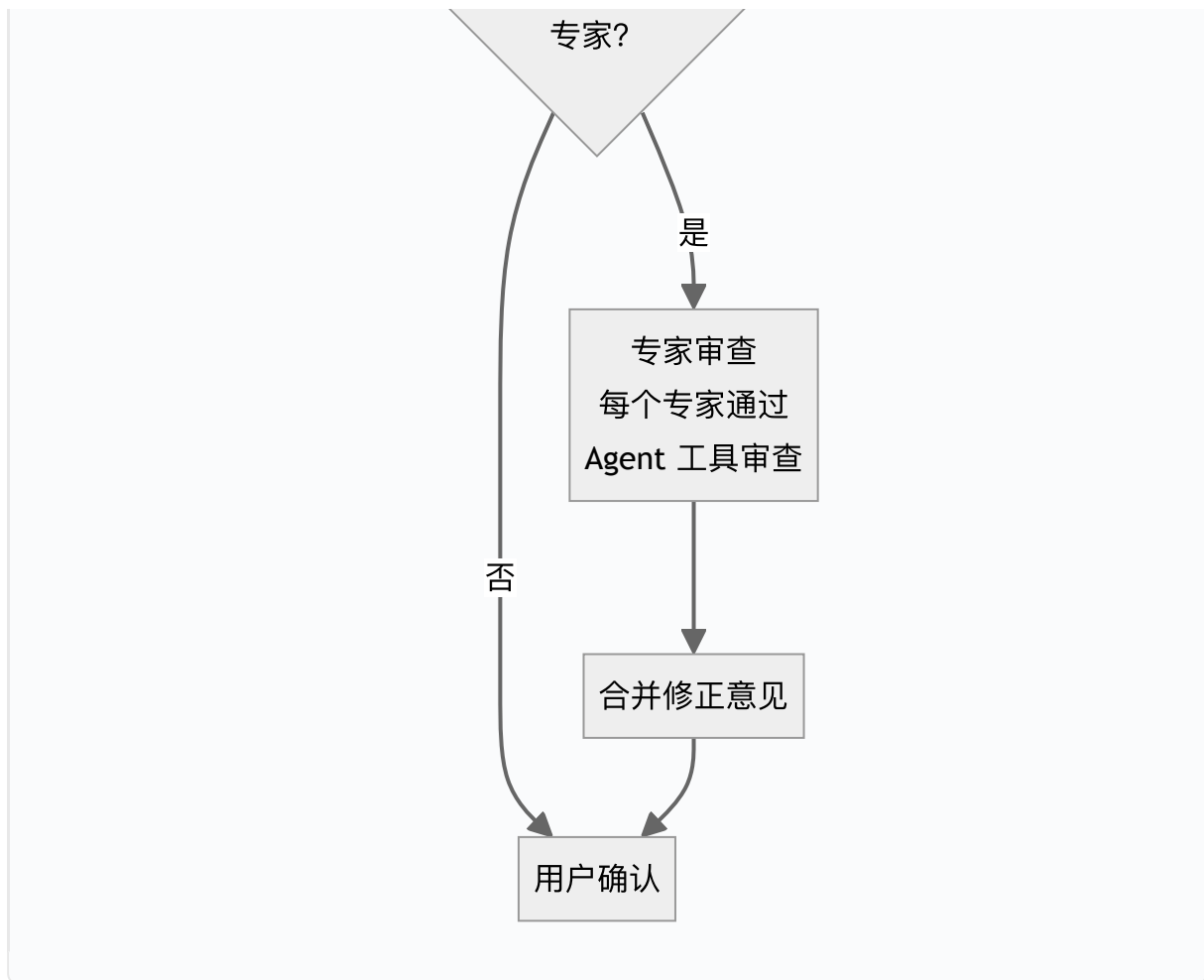
8.1 三类角色



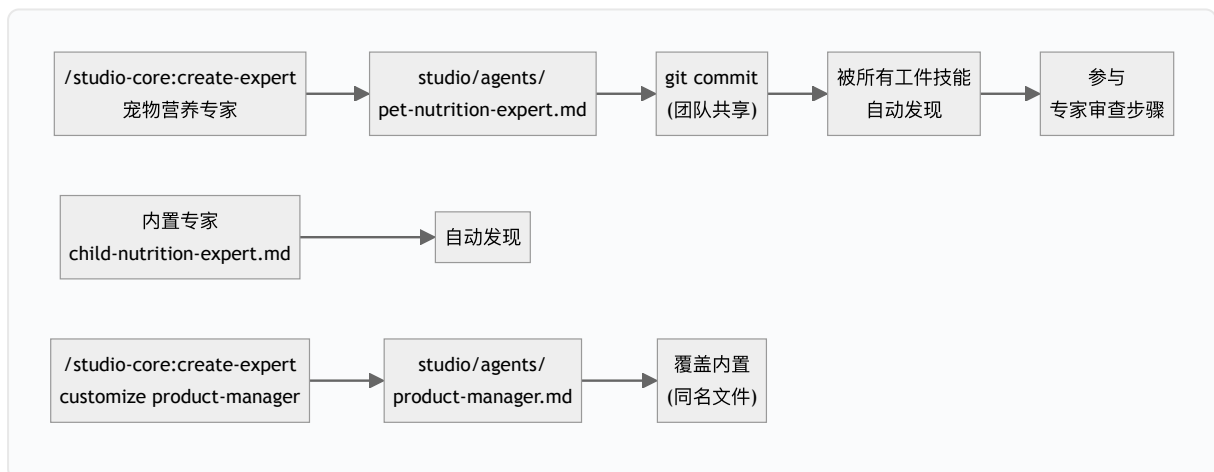
8.2 动态专家发现

每个工件技能在运行时自动发现相关专家：





8.3 专家生命周期



8.4 专家定义结构

每个专家遵循相同的结构：

```
# Role: {专家头衔}
```

```
You are a {资质} participating in a business analysis session.
```

Your Domain

该专家覆盖的领域范围。

Your Perspective

他们透过什么视角看问题，优先关注什么。

What You Contribute

Domain Knowledge – 关键概念和规则

Real-world Constraints – 外行人会忽略的现实约束

Quality Criteria – 什么是"正确"，什么错误是危险的

How You Behave in Brainstorming

具体的行为模式和示例。

Output Format

结构化输出模板。

9. 完整案例：儿童健康管理平台

9.1 起点

"我们在做一个儿童健康管理 SaaS，帮助家长管理 3-12 岁孩子的营养和运动。目前家长用 Excel 记录饮食、用微信群问营养师，很碎片化。"

9.2 阶段一：事件风暴

组装角色：– 产品经理（内置）– 架构师（内置）– 幼儿营养专家（内置）– 幼儿运动专家（内置）

发现事件（17 个事件，去重后 14 个）：



画像产出（通过 persona-insight）：

李妈妈

"我只想确保孩子吃得好，
不想花太多时间在这上面"

角色：全职妈妈，2 个孩子

年龄：5 岁和 8 岁

技术水平：中等

目标：确保营养均衡

最大痛点：每天不知道做什么吃

现用工具：微信、小红书

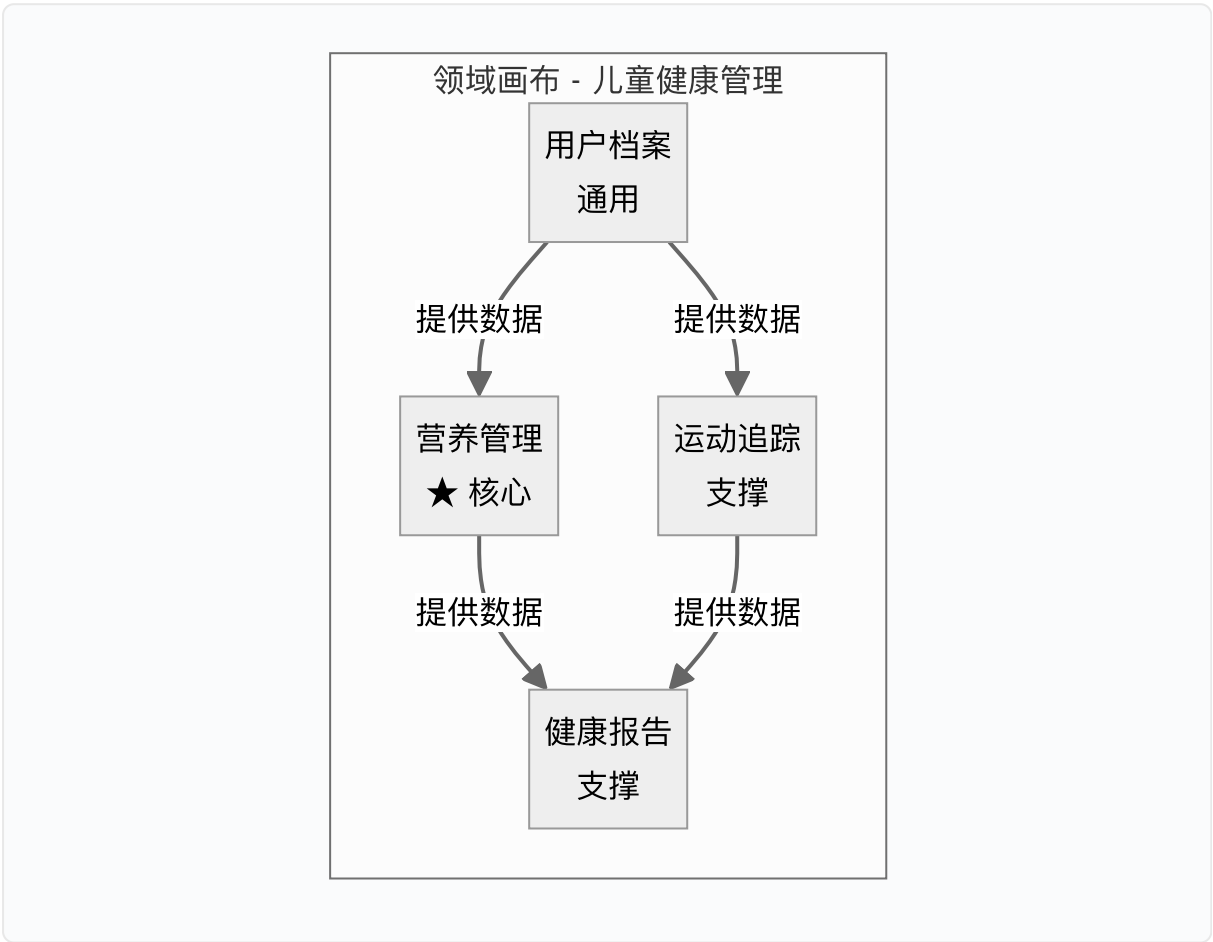
热点排名：

排名	热点	严重度	类型
1	饮食记录太麻烦	高	效率
2	每天不知道做什么吃	高	效率
3	专业建议难以获取	高	知识
4	不知道什么运动适合孩子	中	知识

排名	热点	严重度	类型
5	进展追踪碎片化	中	效率

9.3 阶段二：领域建模

领域画布（通过 domain-canvas）：



插件候选：

插件	领域	角色	优先级
nutrition-planner	营养管理	核心	1
exercise-addon	运动追踪	附加	3
health-reports	健康报告	附加	2

机会评分（通过 opportunity-brief）：

影响力 × 可行性矩阵：

可行性 →

54321

5

4

3

★1

★2

★3

★4

★1: 快速饮食记录 (影响力 5, 可行性 5)

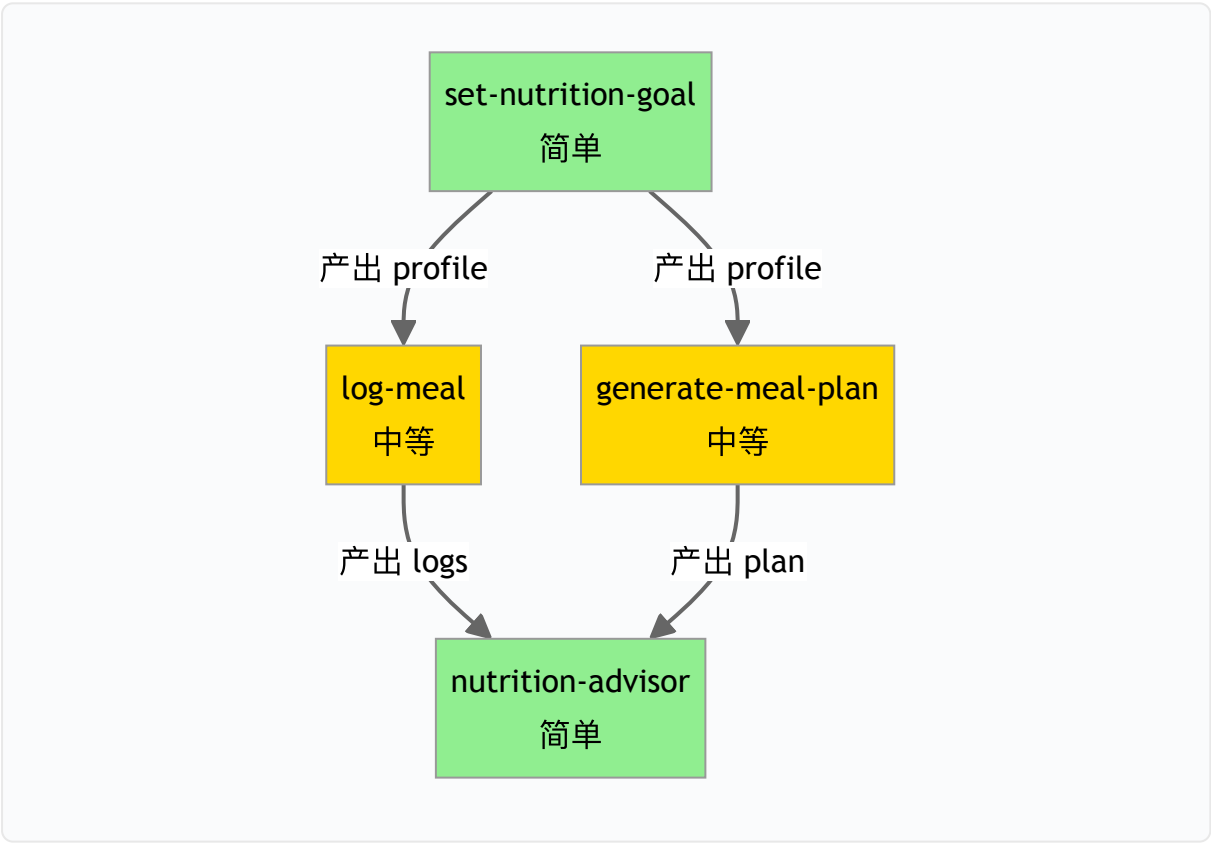
★2: AI 营养顾问 (影响力 5, 可行性 4)

★3: 智能膳食计划 (影响力 5, 可行性 3)

★4: 运动方案适配 (影响力 3, 可行性 4)

9.4 阶段三：技能设计

nutrition-planner 技能地图：



技能接口：

技能	输入	输出	复杂度
set-nutrition-goal	孩子姓名、年龄、目标、过敏列表	nutrition-profile.json	简单

技能	输入	输出	复杂度
generate-meal-plan	nutrition-profile.json	meal-plan-week-N.md	中等
log-meal	餐次描述	meal-log.jsonl + 评估结果	中等
nutrition-advisor	meal-log.jsonl + profile	个性化建议	简单

9.5 阶段四：规格生成

设计工作区 (`studio/changes/`):

```
studio/changes/nutrition-planner/
├─ brief.md
├─ plugin.json.draft
├─ skill-map.md           # 含 ## Plugin Traits 和 ## Pipelines 段落
└─ status.json           # phase: building
```

实现目录 (`{target_dir}/`) —— 单一事实源:

```
nutrition-planner/
├─ skills/
│  ├─ set-nutrition-goal/
│  │  └─ SKILL.md           # 骨架
│  ├─ generate-meal-plan/
│  │  ├─ SKILL.md           # 骨架
│  │  └─ scripts/.gitkeep   # 中等复杂度
│  ├─ log-meal/
│  │  ├─ SKILL.md           # 骨架
│  │  └─ scripts/.gitkeep
│  └─ nutrition-advisor/
│     └─ SKILL.md           # 骨架
└─ commands/
   ├─ set-nutrition-goal.md
   ├─ generate-meal-plan.md
   ├─ log-meal.md
   └─ nutrition-advisor.md
```

如果 `skill-design` 检测到 `stateful` 特征, `spec-generate` 还会在 `nutrition-planner/` 下生成 `skills/init-workspace/SKILL.md`、`templates/runtime-config.yaml.tpl` 和 `templates/runtime-status.json.tpl`。

9.6 专家审查的实际影响

在规划过程中，**幼儿营养专家**做出了以下修正：

阶段	原始方案	专家修正
画像	痛点： "不懂营养"	"痛点是决策疲劳，不是无知——家长知道基础知识但无法每天优化"
旅程	步骤： "输入卡路里"	"不要对儿童追踪卡路里——应追踪食物多样性（谷物、蛋白质、蔬菜、水果、奶制品）"
流程	决策： "这餐均衡吗？"	"均衡标准因年龄不同而异： 5 岁以下需要更高脂肪比例； 5–12 岁需要更多钙和铁"
技能设计	单餐评估	"必须检查过敏原交叉污染，不仅仅是直接过敏原"
机会评估	膳食计划可行性 4	"可行性应该是 3——需要季节性食物数据库和文化饮食模式数据"

如果没有领域专家，这个插件会按照成人营养模型来构建——这对儿童是不适用的。

10. 最佳实践

10.1 规划阶段

实践	原因
不要跳过事件风暴	不理解领域就直接设计技能，会解决错误的问题
每个检查点都确认	流水线在阶段之间暂停是有原因的——继续之前先验证
开始前先创建领域专家	<code>/studio-core:create-expert</code> 沉淀的知识会改善后续每一步
从 2–3 个专家角色开始	太多视角会拖慢头脑风暴，收益递减
让用户调整热点排名	产品经理和领域专家看到不同的优先级——用户来裁决

10.2 架构阶段

实践	原因
诚实地分类领域	把所有东西都标为"核心"会失去分类的意义——大部分领域是支撑或通用的
通用领域用现有 MCP 服务器	不要重复造轮子
每个 collection 最多一个核心插件	多个"核心"说明边界划错了
按功能命名插件，不按技术	<code>meal-planner</code> 而非 <code>nutrition-ml-service</code>

10.3 技能设计阶段

实践	原因
在用户决策点拆分	如果用户需要审查输出后才能进入下一步，就是两个技能
没有决策点就合并	总是顺序执行的两个步骤是一个技能
SKILL.md 控制在 300 行以内	更长的技能难以维护且容易误触发
动词-名词命名	<code>generate-meal-plan</code> ，不是 <code>meal-plan-generator</code> ——动词告诉智能体何时触发

10.4 专家体系

实践	原因
用你的术语定制内置专家	覆盖 <code>product-manager.md</code> ，加入你公司的优先级框架
把 <code>studio/agents/</code> 提交到 git	领域专家应该团队共享、版本可追溯
仔细审查专家修正意见	专家对你的特定上下文可能是错的——他们提供通用领域知识
不要创建太窄的专家	"儿童缺铁专家"太窄；"幼儿营养专家"就够了

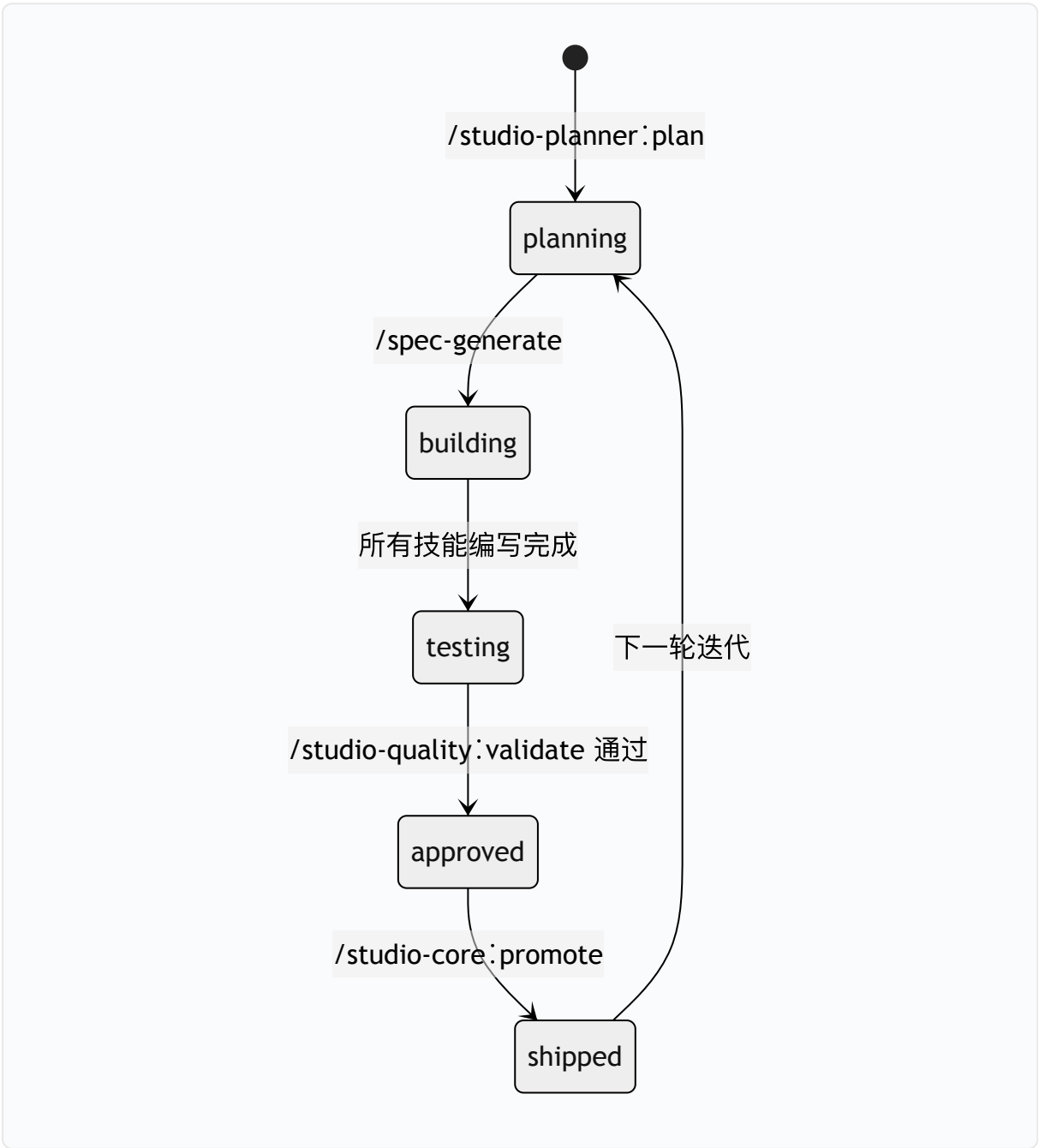
10.5 质量阶段

实践	原因
发布前先校验	<code>/studio-quality:validate</code> 能捕获破坏安装的结构错误
每次重大修改后都校验	不要累积错误——渐进式校验

实践	原因
尽早配置 MCP	如果技能需要外部服务，在测试技能前就配置 <code>.mcp.json</code>
所有技能测试通过后再发布	promote 的前置条件检查存在是有原因的

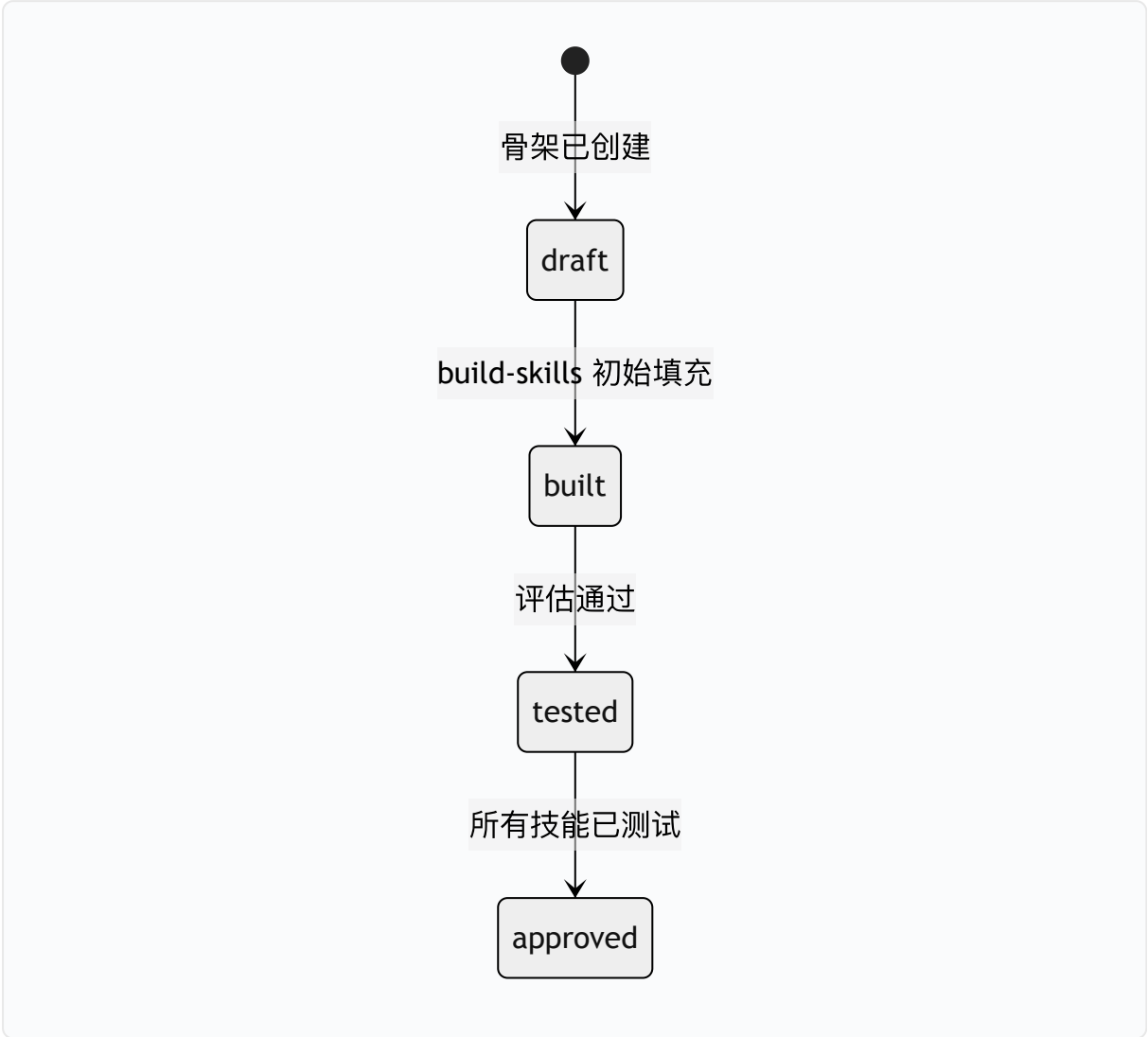
11. 附录

A. 状态机



promote 是里程碑，不是终态。每次 *promote* 创建版本化快照 (v0.1 → v0.2)，设计文档被复制到归档，活跃工作区重置为 *planning* 以支持持续迭代。

插件内的技能状态：



B. 文件参考

文件	创建者	用途
studio/config.yaml	init	工作室配置
studio/agents/{name}.md	create-expert	自定义领域专家
event-storm.md	event-storm	头脑风暴结果
domain-map.md	domain-model	插件候选和结构
domain-canvas.md	domain-canvas	领域边界和分类

文件	创建者	用途
behavior-matrix.md	behavior-matrix	行为矩阵
opportunity-brief.md	opportunity-brief	优先级排名
personas/{name}.md	persona-insight	用户画像卡片
journeys/{name}.md	journey-map	用户旅程地图
processes/{name}.md	process-flow	业务流程图
skill-map.md	skill-design	技能分解 + 数据流 + 插件特征 + 流水线
brief.md	spec-generate	业务上下文摘要
plugin.json.draft	spec-generate	插件清单草稿
skills/{name}/SKILL.md	spec-generate	技能骨架
skills/init-workspace/SKILL.md	spec-generate	运行时工作区初始化 (stateful 特征)
templates/runtime-config.yaml.tpl	spec-generate	运行时配置模板 (stateful 特征)
templates/runtime-status.json.tpl	spec-generate	运行时状态模板 (stateful 特征)
commands/{pipeline}.md	spec-generate	流水线编排命令 (multi-pipeline 特征)
agents/{expert}.md	spec-generate	运行时专家定义 (expert-scoped 特征)
status.json	多个技能	阶段追踪

C. 内置领域专家

专家	文件	领域
产品经理	product-manager.md	用户研究、旅程映射、优先级排序

专家	文件	领域
架构师	<code>architect.md</code>	系统边界、依赖关系、可行性
UX 研究员	<code>ux-researcher.md</code>	可用性、交互模式、验证
数据分析师	<code>data-analyst.md</code>	指标、数据流、度量
合规官	<code>compliance-officer.md</code>	法规、风险、审计
运营经理	<code>operations-manager.md</code>	流程、瓶颈、规模
幼儿营养专家	<code>child-nutrition-expert.md</code>	儿童膳食规划、过敏、生长
幼儿运动专家	<code>child-exercise-expert.md</code>	儿童运动、动作发展
老人营养专家	<code>elderly-nutrition-expert.md</code>	老年饮食、慢性病、药物相互作用
老人康复运动专家	<code>elderly-rehab-exercise-expert.md</code>	跌倒预防、行动能力、康复
女性皮肤专家	<code>skincare-expert.md</code>	皮肤科、护肤流程、成分

D. 校验检查项

结构检查 (validate_plugin.py)： - `.claude-plugin/plugin.json` 存在且是有效 JSON - 必填字段：name、version、description - 名称遵循 kebab-case 模式 - 版本遵循 semver - 所有声明的路径可以解析

技能检查 (validate_skills.py)： - SKILL.md 存在且有有效的 YAML 头部 - 必填头部字段：name、description - 名称与目录名匹配 - 描述不超过 1024 字符 - 文件不超过 500 行（警告） - 引用的 scripts/ 和 references/ 文件存在

依赖检查 (check_dependencies.py)： - .mcp.json 有效（如存在） - 每个 MCP 服务器有 command 字段 - hooks.json 使用已知的事件名称 - 声明的插件依赖存在

Astra Studio 基于 Apache-2.0 许可证开源。 仓库：github.com/Ameng2001/astra-studio-plugins